There are some hidden pitfalls in external declarations and definitions if you use multiple source files. To avoid them, first, define and initialize each external variable only once in the entire set of files:

```
        int     foo  =   0;
```

You can get away with multiple external definitions on UNIX, but not on GCOS, so don't ask for trouble. Multiple initializations are illegal everywhere. Second, at the beginning of any file that contains functions needing a variable whose definition is in some other file, put in an extern declaration, outside of any function:

```
extern  int     foo;

        f1( ) { ... }           etc.
```

#define, #include

C provides a very limited macro facility. You can say

```
        #define name            something
```

and thereafter anywhere ``name'' appears as a token, ``something'' will be substituted. This is particularly useful in parametering the sizes of arrays:

```
        #define ARRAYSIZE       100
                int     arr[ARRAYSIZE];
                 ...
                while( i++ < ARRAYSIZE )...
```

(now we can alter the entire program by changing only the define) or in setting up mysterious constants:

```
        #define SET             01
        #define INTERRUPT       02      /* interrupt bit */
        #define ENABLED 04
         ...
        if( x & (SET | INTERRUPT | ENABLED) ) ...
```

Now we have meaningful words instead of mysterious constants. (The mysterious operators `&' (AND) and `|' (OR) will be covered in the next section.) It's an excellent practice to write programs without any literal constants except in #define statements.

There are several warnings about #define. First, there's no semicolon at the end of a #define; all the text from the name to the end of the line (except for comments) is taken to be the ``something''. When it's put into the text, blanks are placed around it. The other

control word known to C is #include. To include one file in your source at compilation time, say

```
#include "filename"
```

Bit Operators

C has several operators for logical bit-operations. For example,

```
x = x & 0177;
```

forms the bit-wise AND of x and 0177, effectively retaining only the last seven bits of x. Other operators are

```
|       inclusive OR
^       (circumflex) exclusive OR
~       (tilde) 1's complement
!       logical NOT
<<      left shift (as in x<<2)
>>      right shift     (arithmetic on PDP-11;  logical on H6070,
IBM360)
```

Assignment Operators

An unusual feature of C is that the normal binary operators like `+', `-', etc. can be combined with the assignment operator `=' to form new assignment operators. For example,

```
x =- 10;
```

uses the assignment operator `=-' to decrement x by 10, and

```
x =& 0177
```

forms the AND of x and 0177. This convention is a useful notational shortcut, particularly if x is a complicated expression. The classic example is summing an array:

```
for( sum=i=0; i<n; i++ )
        sum =+ array[i];
```

But the spaces around the operator are critical! For

```
x = -10;
```

also decreases x by 10. This is quite contrary to the experience of most programmers. In particular, watch out for things like

```
c=*s++;
y=&x[0];
```

both of which are almost certainly not what you wanted. Newer versions of various compilers are courteous enough to warn you about the ambiguity.

Because all other operators in an expression are evaluated before the assignment operator, the order of evaluation should be watched carefully:

```
x = x<<y | z;
```

means ``shift x left y places, then OR with z, and store in x."

Floating Point

C has single and double precision numbers For example,

```
double sum;
float avg, y[10];
sum = 0.0;
for( i=0; i<n; i++ )
        sum =+ y[i];avg = sum/n;
```

All floating arithmetic is done in double precision. Mixed mode arithmetic is legal; if an arithmetic operator in an expression has both operands `int` or `char`, the arithmetic done is integer, but if one operand is `int` or `char` and the other is `float` or `double`, both operands are converted to `double`. Thus if `i` and `j` are `int` and `x` is `float`,

```
(x+i)/j         converts i and j to float
x + i/j         does i/j integer, then converts
```

Type conversion may be made by assignment; for instance,

```
int m, n;
float x, y;
m = x;
y = n;
```

converts `x` to integer (truncating toward zero), and `n` to floating point.

Floating constants are just like those in Fortran or PL/I, except that the exponent letter is `e' instead of `E'. Thus:

```
pi = 3.14159;
large = 1.23456789e10;
```

`printf` will format floating point numbers: ``%w.df'' in the format string will print the corresponding variable in a field w digits wide, with d decimal places. An e instead of an f will produce exponential notation.

goto and labels

C has a goto statement and labels, so you can branch about the way you used to. But most of the time goto's aren't needed. (How many have we used up to this point?) The code can almost always be more clearly expressed by for/while, if/else, and compound statements.

One use of goto's with some legitimacy is in a program which contains a long loop, where a `while(1)` would be too extended. Then you might write

```
mainloop:
     ...
     goto mainloop;
```

Another use is to implement a `break` out of more than one level of `for` or `while`. goto's can only branch to labels within the same function.

Manipulating Strings

A string is a sequence of characters, with its beginning indicated by a pointer and its end marked by the null character \0. At times, you need to know the length of a string (the number of characters between the start and the end of the string) [5]. This length is obtained with the library function strlen(). Its prototype, in STRING.H, is

```
size_t strlen(char *str);
```

The strcpy() Function

The library function strcpy() copies an entire string to another memory location. Its prototype is as follows:

```
char *strcpy( char *destination, char *source );
```

Before using strcpy(), you must allocate storage space for the destination string.

```
/* Demonstrates strcpy(). */
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

char source[] = "The source string.";
```

```
 main()
 {
    char dest1[80];
    char *dest2, *dest3;

    printf("\nsource: %s", source );

    /* Copy to dest1 is okay because dest1 points to */
    /* 80 bytes of allocated space. */

    strcpy(dest1, source);
    printf("\ndest1:  %s", dest1);

    /* To copy to dest2 you must allocate space. */
    dest2 = (char *)malloc(strlen(source) +1);
    strcpy(dest2, source);
    printf("\ndest2:  %s\n", dest2);

    return(0);
}
source: The source string.
dest1:  The source string.
dest2:  The source string.
```

The strncpy() Function

The strncpy() function is similar to strcpy(), except that strncpy() lets you specify how many characters to copy. Its prototype is

```
char *strncpy(char *destination, char *source, size_t n);
```

```
 /* Using the strncpy() function. */

 #include <stdio.h>
 #include <string.h>

 char dest[] = ".........................";
 char source[] = "abcdefghijklmnopqrstuvwxyz";

 main()
{
    size_t n;

    while (1)
    {
        puts("Enter the number of characters to copy (1-26)");
        scanf("%d", &n);

        if (n > 0 && n< 27)
            break;
    }
```

```
    printf("\nBefore strncpy destination = %s", dest);

    strncpy(dest, source, n);

    printf("\nAfter strncpy destination = %s\n", dest);
    return(0); }

Enter the number of characters to copy (1-26)
15
Before strncpy destination = ........................
After strncpy destination = abcdefghijklmno...........
```

The strdup() Function

The library function strdup() is similar to strcpy(), except that strdup() performs its own memory allocation for the destination string with a call to malloc().The prototype for strdup() is

```
char *strdup( char *source );
```

Using strdup() to copy a string with automatic memory allocation.

```
  /* The strdup() function. */
 #include <stdlib.h>
 #include <stdio.h>
 #include <string.h>

  char source[] = "The source string.";

  main()
  {
     char *dest;

     if ( (dest = strdup(source)) == NULL)
     {
         fprintf(stderr, "Error allocating memory.");
         exit(1);
     }

     printf("The destination = %s\n", dest);
     return(0);
 }
The destination = The source string.
```

The strcat() Function

The prototype of strcat() is

```
char *strcat(char *str1, char *str2);
```

The function appends a copy of str2 onto the end of str1, moving the terminating null character to the end of the new string. You must allocate enough space for str1 to hold the resulting string. The return value of strcat() is a pointer to str1. Following listing demonstrates strcat().

```
 /* The strcat() function. */

 #include <stdio.h>
 #include <string.h>

 char str1[27] = "a";
 char str2[2];

 main()
{
    int n;

    /* Put a null character at the end of str2[]. */

    str2[1] = `\0';

    for (n = 98; n< 123; n++)
    {
        str2[0] = n;
        strcat(str1, str2);
        puts(str1);
    }
     return(0);
}
```

```
ab
abc
abcd
abcde
abcdef
abcdefg
abcdefgh
abcdefghi
abcdefghij
abcdefghijk
abcdefghijkl
abcdefghijklm
abcdefghijklmn
abcdefghijklmno
```

```
abcdefghijklmnop
abcdefghijklmnopq
abcdefghijklmnopqr
abcdefghijklmnopqrs
abcdefghijklmnopqrst
abcdefghijklmnopqrstu
abcdefghijklmnopqrstuv
abcdefghijklmnopqrstuvw
abcdefghijklmnopqrstuvwx
abcdefghijklmnopqrstuvwxy
abcdefghijklmnopqrstuvwxyz
```

Comparing Strings

Strings are compared to determine whether they are equal or unequal. If they are unequal, one string is "greater than" or "less than" the other. Determinations of "greater" and "less" are made with the ASCII codes of the characters. In the case of letters, this is equivalent to alphabetical order, with the one seemingly strange exception that all uppercase letters are "less than" the lowercase letters. This is true because the uppercase letters have ASCII codes 65 through 90 for A through Z, while lowercase a through z are represented by 97 through 122. Thus, "ZEBRA" would be considered to be less than "apple" by these C functions.

The ANSI C library contains functions for two types of string comparisons: comparing two entire strings, and comparing a certain number of characters in two strings.

Comparing Two Entire Strings

The function strcmp() compares two strings character by character. Its prototype is

```
int strcmp(char *str1, char *str2);
```

The arguments str1 and str2 are pointers to the strings being compared. The function's return values are given in Table. Following Listing demonstrates strcmp().

The values returned by strcmp().

Return Value	Meaning
< 0	str1 is less than str2.
0	str1 is equal to str2.
> 0	str1 is greater than str2.

Using strcmp() to compare strings.

```
/* The strcmp() function. */

#include <stdio.h>
#include <string.h>

 main()
 {
     char str1[80], str2[80];
     int x;

    while (1)
    {

        /* Input two strings. */
        printf("\n\nInput the first string, a blank to exit: ");
        gets(str1);

        if ( strlen(str1) == 0 )
            break;

        printf("\nInput the second string: ");
        gets(str2);

        /* Compare them and display the result. */

        x = strcmp(str1, str2);

        printf("\nstrcmp(%s,%s) returns %d", str1, str2, x);
    }
    return(0);
}
```

```
Input the first string, a blank to exit: First string
Input the second string: Second string
strcmp(First string,Second string) returns -1
Input the first string, a blank to exit: test string
Input the second string: test string
strcmp(test string,test string) returns 0
Input the first string, a blank to exit: zebra
Input the second string: aardvark
strcmp(zebra,aardvark) returns 1
Input the first string, a blank to exit:
```

Comparing Partial Strings

The library function strncmp() compares a specified number of characters of one string to another string. Its prototype is

```
int strncmp(char *str1, char *str2, size_t n);
```

The function strncmp() compares n characters of str2 to str1. The comparison proceeds until n characters have been compared or the end of str1 has been reached. The method of comparison and return values are the same as for strcmp(). The comparison is case-sensitive.

Comparing parts of strings with strncmp().

```
 /* The strncmp() function. */

#include <stdio.h>
#include[Sigma]>=tring.h>

 char str1[] = "The first string.";
 char str2[] = "The second string.";
 main()
{
    size_t n, x;

    puts(str1);
    puts(str2);

    while (1)
    {
        puts("\n\nEnter number of characters to compare, 0 to exit.");
        scanf("%d", &n);

        if (n <= 0)
            break;

        x = strncmp(str1, str2, n);

        printf("\nComparing %d characters, strncmp() returns %d.", n, x);
    }
    return(0);
}
```

```
The first string.
The second string.
Enter number of characters to compare, 0 to exit.
3
Comparing 3 characters, strncmp() returns .©]
Enter number of characters to compare, 0 to exit.
6
Comparing 6 characters, strncmp() returns -1.
Enter number of characters to compare, 0 to exit.
0
```

The strchr() Function

The strchr() function finds the first occurrence of a specified character in a string. The prototype is

```
char *strchr(char *str, int ch);
```

The function strchr() searches str from left to right until the character ch is found or the terminating null character is found. If ch is found, a pointer to it is returned. If not, NULL is returned.

When strchr() finds the character, it returns a pointer to that character. Knowing that str is a pointer to the first character in the string, you can obtain the position of the found character by subtracting str from the pointer value returned by strchr(). Following Listing illustrates this. Remember that the first character in a string is at position 0. Like many of C's string functions, strchr() is case-sensitive. For example, it would report that the character F isn't found in the string raffle.

Using strchr() to search a string for a single character.

```
/* Searching for a single character with strchr(). */

 #include <stdio.h>
 #include <string.h>

 main()
 {
     char *loc, buf[80];
     int ch;

    /* Input the string and the character. */

    printf("Enter the string to be searched: ");
    gets(buf);
    printf("Enter the character to search for: ");
    ch = getchar();

    /* Perform the search. */

    loc = strchr(buf, ch);

    if ( loc == NULL )
        printf("The character %c was not found.", ch);
    else
        printf("The character %c was found at position %d.\n",
                ch, loc-buf);
    return(0); }
```

```
Enter the string to be searched: How now Brown Cow?
Enter the character to search for: C
The character C was found at position 14.
```

The strcspn() Function

The library function strcspn() searches one string for the first occurrence of any of the characters in a second string. Its prototype is

```
size_t strcspn(char *str1, char *str2);
```

The function strcspn() starts searching at the first character of str1, looking for any of the individual characters contained in str2. This is important to remember. The function doesn't look for the string str2, but only the characters it contains. If the function finds a match, it returns the offset from the beginning of str1, where the matching character is located. If it finds no match, strcspn() returns the value of strlen(str1). This indicates that the first match was the null character terminating the string

Searching for a set of characters with strcspn().

```
/* Searching with strcspn(). */

 #include <stdio.h>
 #include <string.h>

 main()
 {
     char  buf1[80], buf2[80];
     size_t loc;

    /* Input the strings. */

    printf("Enter the string to be searched: ");
    gets(buf1);
    printf("Enter the string containing target characters: ");
    gets(buf2);

    /* Perform the search. */

    loc = strcspn(buf1, buf2);

    if ( loc ==  strlen(buf1) )
        printf("No match was found.");
    else
        printf("The first match was found at position %d.\n", loc);
    return(0);
}
```

```
Enter the string to be searched: How now Brown Cow?
Enter the string containing target characters: Cat
The first match was found at position 14.
```

The strpbrk() Function

The library function strpbrk() is similar to strcspn(), searching one string for the first occurrence of any character contained in another string. It differs in that it doesn't include the terminating null characters in the search. The function prototype is

```
char *strpbrk(char *str1, char *str2);
```

The function strpbrk() returns a pointer to the first character in str1 that matches any of the characters in str2. If it doesn't find a match, the function returns NULL. As previously explained for the function strchr(), you can obtain the offset of the first match in str1 by subtracting the pointer str1 from the pointer returned by strpbrk() (if it isn't NULL, of course).

The strstr() Function

The final, and perhaps most useful, C string-searching function is strstr(). This function searches for the first occurrence of one string within another, and it searches for the entire string, not for individual characters within the string. Its prototype is

```
char *strstr(char *str1, char *str2);
```

The function strstr() returns a pointer to the first occurrence of str2 within str1. If it finds no match, the function returns NULL. If the length of str2 is 0, the function returns str1. When strstr() finds a match, you can obtain the offset of str2 within str1 by pointer subtraction, as explained earlier for strchr(). The matching procedure that strstr() uses is case-sensitive.

Using strstr() to search for one string within another.

```
/* Searching with strstr(). */

#include <stdio.h>
#include <string.h>

main()
{
    char *loc, buf1[80], buf2[80];

   /* Input the strings. */

   printf("Enter the string to be searched: ");
```

```
     gets(buf1);
     printf("Enter the target string: ");
     gets(buf2);

     /* Perform the search. */

     loc = strstr(buf1, buf2);

     if ( loc ==  NULL )
         printf("No match was found.\n");
     else
         printf("%s was found at position %d.\n", buf2, loc-buf1);
     return(0);}
Enter the string to be searched: How now brown cow?
Enter the target string: cow
Cow was found at position 14.
```

The strrev() Function

The function strrev() reverses the order of all the characters in a string (Not ANSI Standard). Its prototype is

```
char *strrev(char *str);
```

The order of all characters in str is reversed, with the terminating null character remaining at the end.

String-to-Number Conversions

Sometimes you will need to convert the string representation of a number to an actual numeric variable. For example, the string "123" can be converted to a type int variable with the value 123. Three functions can be used to convert a string to a number. They are explained in the following sections; their prototypes are in STDLIB.H.

The atoi() Function

The library function atoi() converts a string to an integer. The prototype is

```
int atoi(char *ptr);
```

The function atoi() converts the string pointed to by ptr to an integer. Besides digits, the string can contain leading white space and a + or -- sign. Conversion starts at the beginning

of the string and proceeds until an unconvertible character (for example, a letter or punctuation mark) is encountered. The resulting integer is returned to the calling program. If it finds no convertible characters, atoi() returns 0. Table lists some examples.

String-to-number conversions with atoi().

String	Value Returned by atoi()
"157"	157
"-1.6"	-1
"+50x"	50
"twelve"	0
"x506"	0

The atol() Function

The library function atol() works exactly like atoi(), except that it returns a type long. The function prototype is

```
long atol(char *ptr);
```

The atof() Function

The function atof() converts a string to a type double. The prototype is

```
double atof(char *str);
```

The argument str points to the string to be converted. This string can contain leading white space and a + or -- character. The number can contain the digits 0 through 9, the decimal point, and the exponent indicator E or e. If there are no convertible characters, atof() returns 0. Table 17.3 lists some examples of using atof().

String-to-number conversions with atof().

String	Value Returned by atof()
"12"	12.000000
"-0.123"	-0.123000
"123E+3"	123000.000000
"123.1e-5"	0.001231

Character Test Functions

The header file CTYPE.H contains the prototypes for a number of functions that test characters, returning TRUE or FALSE depending on whether the character meets a certain condition. For example, is it a letter or is it a numeral? The is*xxxx*() functions are actually macros, defined in CTYPE.H.

The is*xxxx*() macros all have the same prototype:

```
int isxxxx(int ch);
```

In the preceding line, ch is the character being tested. The return value is TRUE (nonzero) if the condition is met or FALSE (zero) if it isn't. Table lists the complete set of is*xxxx*() macros.

The isxxxx() macros.

Macro	Action
isalnum()	Returns TRUE if ch is a letter or a digit.
isalpha()	Returns TRUE if ch is a letter.
isascii()	Returns TRUE if ch is a standard ASCII character (between 0 and 127).
iscntrl()	Returns TRUE if ch is a control character.
isdigit()	Returns TRUE if ch is a digit.
isgraph()	Returns TRUE if ch is a printing character (other than a space).
islower()	Returns TRUE if ch is a lowercase letter.
isprint()	Returns TRUE if ch is a printing character (including a space).
ispunct()	Returns TRUE if ch is a punctuation character.
isspace()	Returns TRUE if ch is a whitespace character (space, tab, vertical tab, line feed, form feed, or carriage return).
isupper()	Returns TRUE if ch is an uppercase letter.
isxdigit()	Returns TRUE if ch is a hexadecimal digit (0 through 9, a through f, A through F).

You can do many interesting things with the character-test macros. One example is the function get_int(), shown in Listing. This function inputs an integer from stdin and returns it as a type int variable. The function skips over leading white space and returns 0 if the first nonspace character isn't a numeric character.

Using the isxxxx() macros to implement a function that inputs an integer.

```
/* Using character test macros to create an integer */
/* input function. */
 #include <stdio.h>
```

```
  #include <ctype.h>

 int get_int(void);

 main()
{
    int x;
    x =  get_int();
    printf("You entered %d.\n", x);
}

int get_int(void)
{
    int ch, i, sign = 1;

    while ( isspace(ch = getchar()) );

    if (ch != `-' && ch != `+' && !isdigit(ch) && ch != EOF)
    {
        ungetc(ch, stdin);
        return 0;
    }

    /* If the first character is a minus sign, set */
    /* sign accordingly. */

    if (ch == `-')
        sign = -1;

    /* If the first character was a plus or minus sign, */
    /* get the next character. */

    if (ch == `+' || ch == `-')
        ch = getchar();

    /* Read characters until a nondigit is input. Assign */
    /* values, multiplied by proper power of 10, to i. */

    for (i = 0; isdigit(ch); ch = getchar() )
        i = 10 * i + (ch - `0');

    /* Make result negative if sign is negative. */

    i *= sign;

    /* If EOF was not encountered, a nondigit character */
    /* must have been read in, so unget it. */
    if (ch != EOF)
        ungetc(ch, stdin);

    /* Return the input value. */

    return i;
}
```

```
-100
You entered -100.
abc3.145
You entered 0.
9 9 9
You entered 9.
2.5
You entered 2.
```

Mathematical Functions

The C standard library contains a variety of functions that perform mathematical operations. Prototypes for the mathematical functions are in the header file MATH.H. The math functions all return a type double. For the trigonometric functions, angles are expressed in radians. Remember, one radian equals 57.296 degrees, and a full circle (360 degrees) contains 2p radians.

Trigonometric Functions

Function	Prototype	Description
acos()	double acos(double x)	Returns the arccosine of its argument. The argument must be in the range -1 <= x <= 1, and the return value is in the range 0 <= acos <= p.
asin()	double asin(double x)	Returns the arcsine of its argument. The argument must be in the range -1 <= x <= 1, and the return value is in the range -p/2 <= asin <= p/2.
atan()	double atan(double x)	Returns the arctangent of its argument. The return value is in the range -p/2 <= atan <= p/2.
atan2()	double atan2(double x, double y)	Returns the arctangent of x/y. The value returned is in the range -p <= atan2 <= p.
cos()	double cos(double x)	Returns the cosine of its argument.
sin()	double sin(double x)	Returns the sine of its argument.
tan()	double tan(double x)	Returns the tangent of its argument.

Exponential and Logarithmic Functions

Function	Prototype	Description
exp()	double exp(double x)	Returns the natural exponent of its argument, that is, e^x where e equals 2.7182818284590452354.
log()	double log(double x)	Returns the natural logarithm of its argument. The argument must be greater than 0.
log10()	double log10	Returns the base-10 logarithm of its argument. The argument must be greater than 0.

Hyperbolic Functions

Function	Prototype	Description
cosh()	double cosh(double x)	Returns the hyperbolic cosine of its argument.
sinh()	double sinh(double x)	Returns the hyperbolic sine of its argument.
tanh()	double tanh(double x)	Returns the hyperbolic tangent of its argument.

Other Mathematical Functions

Function	Prototype	Description
sqrt()	double sqrt(double x)	Returns the square root of its argument. The argument must be zero or greater.
ceil()	double ceil(double x)	Returns the smallest integer not less than its argument. For example, ceil(4.5) returns 5.0, and ceil(-4.5) returns -4.0. Although ceil() returns an integer value, it is returned as a type double.
abs()	int abs(int x)	Returns the absolute
labs()	long labs(long x)	value of their arguments.
floor()	double floor(double x)	Returns the largest integer not greater than its argument. For example, floor(4.5) returns 4.0, and floor(-4.5) returns -5.0.
modf()	double modf(double x, double *y)	Splits x into integral and fractional parts, each with the same sign as x. The fractional part is returned by the function, and the integral part is assigned to *y.
pow()	double pow(double x, double y)	Returns xy. An error occurs if x == 0 and y <= 0, or if x < 0 and y is not an integer.

CHAPTER 4 ESSENTIAL DATA STRUCTRURES

In every algorithm, there is a need to store data. Ranging from storing a single value in a single variable, to more complex data structures. In programming contests, there are several aspect of data structures to consider when selecting the proper way to represent the data for a problem. This chapter will give you some guidelines and list some basic data structures to start with.[2]

Will it work?

If the data structures won't work, it's not helpful at all. Ask yourself what questions the algorithm will need to be able to ask the data structure, and make sure the data structure can handle it. If not, then either more data must be added to the structure, or you need to find a different representation.

Can I code it?

If you don't know or can't remember how to code a given data structure, pick a different one. Make sure that you have a good idea how each of the operations will affect the structure of the data.

Another consideration here is memory. Will the data structure fit in the available memory? If not, compact it or pick a new one. Otherwise, it is already clear from the beginning that it won't work.

Can I code it in time? or has my programming language support it yet?

As this is a timed contest, you have three to five programs to write in, say, five hours. If it'll take you an hour and a half to code just the data structure for the first problem, then you're almost certainly looking at the wrong structure.

Another very important consideration is, whether your programming language has actually provide you the required data structure. For C++ programmers, STL has a wide options of a very good built-in data structure, what you need to do is to master them, rather than trying to built your own data structure.

In contest time, this will be one of the winning strategy. Consider this scenario. All teams are given a set of problems. Some of them required the usage of 'stack'. The best programmer from team A directly implement the best known stack implementation, he need 10 minutes to do so. Surprisingly for them, other teams type in only two lines:

"#include <stack>" and "std::stack<int> s;", can you see who have 10 minutes advantage now?

Can I debug it?

It is easy to forget this particular aspect of data structure selection. Remember that a program is useless unless it works. Don't forget that debugging time is a large portion of the contest time, so include its consideration in calculating coding time.

What makes a data structure easy to debug? That is basically determined by the following two properties.

1. State is easy to examine. The smaller, more compact the representation, in general, the easier it is to examine. Also, statically allocated arrays are **much** easier to examine than linked lists or even dynamically allocated arrays.

2. State can be displayed easily. For the more complex data structures, the easiest way to examine them is to write a small routine to output the data. Unfortunately, given time constraints, you'll probably want to limit yourself to text output. This means that structures like trees and graphs are going to be difficult to examine.

Is it fast?

The usage of more sophisticated data structure will reduce the amount of overall algorithm complexity. It will be better if you know some advanced data structures.

Things to Avoid: Dynamic Memory

In general, you should avoid dynamic memory, because:

1. It is too easy to make mistakes using dynamic memory. Overwriting past allocated memory, not freeing memory, and not allocating memory are only some of the mistakes that are introduced when dynamic memory is used. In addition, the failure modes for these errors are such that it's hard to tell where the error occurred, as it's likely to be at a (potentially much later) memory operation.

2. It is too hard to examine the data structure's contents. The interactive development environments available don't handle dynamic memory well, especially for C. Consider parallel arrays as an alternative to dynamic memory. One way to do a linked list, where instead of keeping a next point, you keep a second array, which has the index of the next

element. Sometimes you may have to dynamically allocate these, but as it should only be done once, it's much easier to get right than allocating and freeing the memory for each insert and delete.

All of this notwithstanding, sometimes dynamic memory is the way to go, especially for large data structures where the size of the structure is not known until you have the input.

Things to Avoid: Coolness Factor

Try not to fall into the 'coolness' trap. You may have just seen the neatest data structure, but remember:

1. Cool ideas that don't work aren't.
2. Cool ideas that'll take forever to code aren't, either

It's much more important that your data structure and program work than how impressive your data structure is.

Basic Data Structures

There are five basic data structures: arrays, linked lists, stacks, queues, and deque (pronounced deck, a double ended queue). It will not be discussed in this section, go for their respective section.

Arrays

Array is the most useful data structures, in fact, this data structure will almost always used in all contest problems.Lets look at the good side first: if index is known, searching an element in an array is very fast, O(1), this good for looping/iteration. Array can also be used to implement other sophisticated data structures such as stacks, queues, hash tables.However, being the easiest data structure doesn't mean that array is efficient. In some cases array can be very inefficient. Moreover, in standard array, the size is fixed. If you don't know the input size beforehand, it may be wiser to use vector (a resizable array). Array also suffer a very slow insertion in ordered array, another slow searching in unordered array, and unavoidable slow deletion because you have to shift all elements.

Search	O(n/2) comparisons	O(n) comparisons
Insertion	No comparison, O(1)	No comparisons, O(1)
Deletion	O(n/2) comparisons,O(n/2) moves	O(n) comparisons more than O(n/2) moves

We commonly use one-dimensional array for standard use and two-dimensional array to represent matrices, board, or anything that two-dimensional. Three-dimensional is rarely used to model 3D situations.

Row major, cache-friendly, use this method to access all items in array sequentially.

```
for (i=0; i<numRow; i++)     // ROW FIRST = EFFECTIVE
   for (j=0; j<numCol; j++)
      array[i][j] = 0;
```

Column major, NOT cache-friendly, DO NOT use this method or you'll get very poor performance. Why? you will learn this in Computer Architecture course. For now, just take note that computer access array data in row major.

```
for (j=0; j<numCol; j++)     // COLUMN FIRST = INEFFECTIVE
   for (i=0; i<numRow; i++)
      array[i][j] = 0;
```

Vector Trick - A resizable array

Static array has a drawback when the size of input is not known beforehand. If that is the case, you may want to consider vector, a resizable array.

There are other data structures which offer much more efficient resizing capability such as Linked List, etc.

TIPS: UTILIZING C++ VECTOR STL

C++ STL has vector template for you.

```
#include <stdio.h>
#include <vector>
#include <algorithm>

using namespace std;

void main() {
  // just do this, write vector<the type you want,
  // in this case, integer> and the vector name
  vector<int> v;

  // try inserting 7 different integers, not ordered
  v.push_back(3); v.push_back(1); v.push_back(2);
  v.push_back(7); v.push_back(6); v.push_back(5);
  v.push_back(4);
```

```
    // to access the element, you need an iterator...
    vector<int>::iterator i;

    printf("Unsorted version\n");
    // start with 'begin', end with 'end', advance with i++
    for (i = v.begin(); i!= v.end(); i++)
      printf("%d ",*i); // iterator's pointer hold the value
    printf("\n");

    sort(v.begin(),v.end()); // default sort, ascending

    printf("Sorted version\n");
    for (i = v.begin(); i!= v.end(); i++)
      printf("%d ",*i); // iterator's pointer hold the value
    printf("\n");
  }
```

Linked List

Motivation for using linked list: Array is static and even though it has O(1) access time if index is known, Array must shift its elements if an item is going to be inserted or deleted and this is absolutely inefficient. Array cannot be resized if it is full (see resizable array - vector for the trick but slow resizable array).

Linked list can have a very fast insertion and deletion. The physical location of data in linked list can be anywhere in the memory but each node must know which part in memory is the next item after them.

Linked list can be any big as you wish as long there is sufficient memory. The side effect of Linked list is there will be wasted memory when a node is "deleted" (only flagged as deleted). This wasted memory will only be freed up when garbage collector doing its action (depends on compiler / Operating System used).

Linked list is a data structure that is commonly used because of it's dynamic feature. Linked list is not used for fun, it's very complicated and have a tendency to create run time memory access error). Some programming languages such as Java and C++ actually support Linked list implementation through API (Application Programming Interface) and STL (Standard Template Library).

Linked list is composed of a data (and sometimes pointer to the data) and a pointer to next item. In Linked list, you can only find an item through complete search from head until it found the item or until tail (not found). This is the bad side for Linked list, especially for a very long list. And for insertion in Ordered Linked List, we have to search for appropriate place using Complete Search method, and this is slow too. (There are some tricks to improve searching in Linked List, such as remembering references to specific nodes, etc).

Variations of Linked List

With tail pointer
Instead of standard head pointer, we use another pointer to keep track the last item. This is useful for queue-like structures since in Queue, we enter Queue from rear (tail) and delete item from the front (head).

With dummy node (sentinels)
This variation is to simplify our code (I prefer this way), It can simplify empty list code and inserting to the front of the list code.

Doubly Linked List
Efficient if we need to traverse the list in both direction (forward and backward). Each node now have 2 pointers, one point to the next item, the other one point to the previous item. We need dummy head & dummy tail for this type of linked list.

TIPS: UTILIZING C++ LIST STL

```
A demo on the usage of STL list. The underlying data structure is a
doubly link list.
```

```
#include <stdio.h>

// this is where list implementation resides
#include <list>

// use this to avoid specifying "std::" everywhere
using namespace std;

// just do this, write list<the type you want,
// in this case, integer> and the list name
list<int> l;
list<int>::iterator i;

void print() {
  for (i = l.begin(); i != l.end(); i++)
    printf("%d ",*i); // remember... use pointer!!!
  printf("\n");
}

void main() {
  // try inserting 8 different integers, has duplicates
  l.push_back(3); l.push_back(1); l.push_back(2);
  l.push_back(7); l.push_back(6); l.push_back(5);
  l.push_back(4); l.push_back(7);
  print();
```

```
  l.sort(); // sort the list, wow sorting linked list...
  print();

  l.remove(3); // remove element '3' from the list
  print();

  l.unique(); // remove duplicates in SORTED list!!!
  print();

  i = l.begin(); // set iterator to head of the list
  i++; // 2nd node of the list
  l.insert(i,1,10); // insert 1 copy of '10' here
  print();
}
```

Stack

A data structures which only allow insertion (push) and deletion (pop) from the top only. This behavior is called Last In First Out (LIFO), similar to normal stack in the real world.

Important stack operations

1. Push (C++ STL: push())
   Adds new item at the top of the stack.
2. Pop (C++ STL: pop())
   Retrieves and removes the top of a stack.
3. Peek (C++ STL: top())
   Retrieves the top of a stack without deleting it.
4. IsEmpty (C++ STL: empty())
   Determines whether a stack is empty.

Some stack applications

1. To model "real stack" in computer world: Recursion, Procedure Calling, etc.
2. Checking palindrome (although checking palindrome using Queue & Stack is 'stupid').
3. To read an input from keyboard in text editing with backspace key.
4. To reverse input data, (another stupid idea to use stack for reversing data).
5. Checking balanced parentheses.
6. Postfix calculation.
7. Converting mathematical expressions. Prefix, Infix, or Postfix.

Some stack implementations

1. Linked List with head pointer only (Best)
2. Array
3. Resizeable Array

TIPS: UTILIZING C++ STACK STL

Stack is not difficult to implement.Stack STL's implementation is very efficient, even though it will be slightly slower than your custom made stack.

```
#include <stdio.h>
#include <stack>

using namespace std;

void main() {
  // just do this, write stack<the type you want,
  // in this case, integer> and the stack name
  stack<int> s;

  // try inserting 7 different integers, not ordered
  s.push(3); s.push(1); s.push(2);
  s.push(7); s.push(6); s.push(5);
  s.push(4);

  // the item that is inserted first will come out last
  // Last In First Out (LIFO) order...
  while (!s.empty()) {
    printf("%d ",s.top());
    s.pop();
  }
  printf("\n");}
```

Queue

A data structures which only allow insertion from the back (rear), and only allow deletion from the head (front). This behavior is called First In First Out (FIFO), similar to normal queue in the real world.

Important queue operations:

1. Enqueue (C++ STL: push())
   Adds new item at the back (rear) of a queue.
2. Dequeue (C++ STL: pop())

Retrieves and removes the front of a queue at the back (rear) of a queue.

3. Peek (C++ STL: top())
   Retrieves the front of a queue without deleting it.
4. IsEmpty (C++ STL: empty())
   Determines whether a queue is empty.

TIPS: UTILIZING C++ QUEUE STL

Standard queue is also not difficult to implement. Again, why trouble yourself, just use C++ queue STL.

```
#include <stdio.h>
#include <queue>

// use this to avoid specifying "std::" everywhere
using namespace std;

void main() {
  // just do this, write queue<the type you want,
  // in this case, integer> and the queue name
  queue<int> q;

  // try inserting 7 different integers, not ordered
  q.push(3); q.push(1); q.push(2);
  q.push(7); q.push(6); q.push(5);
  q.push(4);

  // the item that is inserted first will come out first
  // First In First Out (FIFO) order...
  while (!q.empty()) {
    // notice that this is not "top()" !!!
    printf("%d ",q.front());
    q.pop();
  }
 printf("\n");}
```

CHAPTER 5 INPUT/OUTPUT TECHNIQUES

In all programming contest, or more specifically, in all useful program, you need to read in input and process it. However, the input data can be as nasty as possible, and this can be very troublesome to parse.[2]

If you spent too much time in coding how to parse the input efficiently and you are using C/C++ as your programming language, then this tip is for you. Let's see an example:

How to read the following input:

```
1 2 2 3 1 2 3 1 2
```

The fastest way to do it is:

```
#include <stdio.h>
int N;
void main() {
while(scanf("%d",&N==1)){
    process N…  }
}
```

There are N lines, each lines always start with character '0' followed by '.', then unknown number of digits x, finally the line always terminated by three dots "...".

```
N
0.xxxx...
```

The fastest way to do it is:

```
#include <stdio.h>
char digits[100];


void main() {
scanf("%d",&N);
for (i=0; i<N; i++) {
    scanf("0.%[0-9]...",&digits); // surprised?
    printf("the digits are 0.%s\n",digits);
  }
}
```

This is the trick that many C/C++ programmers doesn't aware of. Why we said C++ while scanf/printf is a standard C I/O routines? This is because many C++ programmers

"forcing" themselves to use cin/cout all the time, without realizing that scanf/printf can still be used inside all C++ programs.

Mastery of programming language I/O routines will help you a lot in programming contests.

Advanced use of printf() and scanf()

Those who have forgotten the advanced use of printf() and scanf(), recall the following examples:

```
scanf("%[ABCDEFGHIJKLMNOPQRSTUVWXYZ]",&line); //line is a string
```

This scanf() function takes only uppercase letters as input to line and any other characters other than A..Z terminates the string. Similarly the following scanf() will behave like gets():

```
scanf("%[^\n]",line); //line is a string
```

Learn the default terminating characters for scanf(). Try to read all the advanced features of scanf() and printf(). This will help you in the long run.

Using new line with scanf()

If the content of a file (input.txt) is

```
abc
def
```

And the following program is executed to take input from the file:

```
char input[100],ch;
void main(void)
{
    freopen("input.txt","rb",stdin);
    scanf("%s",&input);
    scanf("%c",&ch);
}
```

The following is a slight modification to the code:

```
char input[100],ch;
void main(void)
{
    freopen("input.txt","rb",stdin);
    scanf("%s\n",&input);
    scanf("%c",&ch);
}
```

What will be their value now? The value of ch will be '\n' for the first code and 'd' for the second code.

Be careful about using gets() and scanf() together !

You should also be careful about using gets() and scanf() in the same program. Test it with the following scenario. The code is:

```
scanf("%s\n",&dummy);
gets(name);
```

And the input file is:

```
ABCDEF
bbbbbXXX
```

What do you get as the value of name? "XXX" or "bbbbbXXX" (Here, "b" means blank or space)

Multiple input programs

"Multiple input programs" are an invention of the online judge. The online judge often uses the problems and data that were first presented in live contests. Many solutions to problems presented in live contests take a single set of data, give the output for it, and terminate. This does not imply that the judges will give only a single set of data. The judges actually give multiple files as input one after another and compare the corresponding output files with the judge output. However, the Valladolid online judge gives only one file as input. It inserts all the judge inputs into a single file and at the top of that file, it writes how many sets of inputs there are. This number is the same as the number of input files the contest judges used. A blank line now separates each set of data. So the structure of the input file for multiple input program becomes:

```
Integer N     //denoting the number of sets of input
--blank line---
input set 1 //As described in the problem statement
--blank line---

input set 2 //As described in the problem statement
--blank line---
input set 3 //As described in the problem statement
--blank line---
.
.
.
--blank line---
input set n  //As described in the problem statement
--end of file--
```

Note that there should be no blank after the last set of data. The structure of the output file for a multiple input program becomes:

```
Output for set 1 //As described in the problem statement
--Blank line---
Output for set 2 //As described in the problem statement
--Blank line---
Output for set 3 //As described in the problem statement
--Blank line---
.
.
.
--blank line---
Output for set n //As described in the problem statement
--end of file--
```

The USU online judge does not have multiple input programs like Valladolid. It prefers to give multiple files as input and sets a time limit for each set of input.

Problems of multiple input programs

There are some issues that you should consider differently for multiple input programs. Even if the input specification says that the input terminates with the end of file (EOF), each set of input is actually terminated by a blank line, except for the last one, which is terminated by the end of file. Also, be careful about the initialization of variables. If they are not properly initialized, your program may work for a single set of data but give correct output for multiple sets of data. All global variables are initialized to their corresponding zeros. [6]

CHAPTER 6 BRUTE FORCE METHOD

This is the most basic problem solving technique. Utilizing the fact that computer is actually very fast.

Complete search exploits the brute force, straight-forward, try-them-all method of finding the answer. This method should almost always be the first algorithm/solution you consider. If this works within time and space constraints, then do it: it's easy to code and usually easy to debug. This means you'll have more time to work on all the hard problems, where brute force doesn't work quickly enough.

Party Lamps

You are given N lamps and four switches. The first switch toggles all lamps, the second the even lamps, the third the odd lamps, and last switch toggles lamps 1,4,7,10,...

Given the number of lamps, N, the number of button presses made (up to 10,000), and the state of some of the lamps (e.g., lamp 7 is off), output all the possible states the lamps could be in.

Naively, for each button press, you have to try 4 possibilities, for a total of 4^10000 (about 10^6020), which means there's no way you could do complete search (this particular algorithm would exploit recursion).

Noticing that the order of the button presses does not matter gets this number down to about 10000^4 (about 10^16), still too big to completely search (but certainly closer by a factor of over 10^6000).

However, pressing a button twice is the same as pressing the button no times, so all you really have to check is pressing each button either 0 or 1 times. That's only 2^4 = 16 possibilities, surely a number of iterations solvable within the time limit.

The Clocks

A group of nine clocks inhabits a 3 x 3 grid; each is set to 12:00, 3:00, 6:00, or 9:00. Your goal is to manipulate them all to read 12:00. Unfortunately, the only way you can manipulate the clocks is by one of nine different types of move, each one of which rotates a certain subset of the clocks 90 degrees clockwise. Find the shortest sequence of moves which returns all the clocks to 12:00.

The 'obvious' thing to do is a recursive solution, which checks to see if there is a solution of 1 move, 2 moves, etc. until it finds a solution. This would take 9^k time, where k is the number of moves. Since k might be fairly large, this is not going to run with reasonable time constraints.

Note that the order of the moves does not matter. This reduces the time down to k^9, which isn't enough of an improvement.

However, since doing each move 4 times is the same as doing it no times, you know that no move will be done more than 3 times. Thus, there are only 49 possibilities, which is only 262,072, which, given the rule of thumb for run-time of more than 10,000,000 operations in a second, should work in time. The brute-force solution, given this insight, is perfectly adequate.

It is beautiful, isn't it? If you have this kind of reasoning ability. Many seemingly hard problems is eventually solvable using brute force.

Recursion

Recursion is cool. Many algorithms need to be implemented recursively. A popular combinatorial brute force algorithm, backtracking, usually implemented recursively. After highlighting it's importance, lets study it.

Recursion is a function/procedure/method that calls itself again with a smaller range of arguments (break a problem into simpler problems) or with different arguments (it is useless if you use recursion with the same arguments). It keeps calling itself until something that is so simple / simpler than before (which we called base case), that can be solved easily or until an exit condition occurs.

A recursion must stop (actually, all program must terminate). In order to do so, a valid base case or end-condition must be reached. Improper coding will leads to stack overflow error.

You can determine whether a function recursive or not by looking at the source code. If in a function or procedure, it calls itself again, it's recursive type.

When a method/function/procedure is called:
– caller is suspended,
– "state" of caller saved,
– new space allocated for variables of new method.

Recursion is a powerful and elegant technique that can be used to solve a problem by solving a smaller problem of the same type. Many problems in Computer Science involve recursion and some of them are naturally recursive.

If one problem can be solved in both way (recursive or iterative), then choosing iterative version is a good idea since it is faster and doesn't consume a lot of memory. Examples: Factorial, Fibonacci, etc.

However, there are also problems that are can only be solved in recursive way or more efficient in recursive type or when it's iterative solutions are difficult to conceptualize. Examples: Tower of Hanoi, Searching (DFS, BFS), etc.

Types of recursion

There are 2 types of recursion, Linear recursion and multiple branch (Tree) recursion.

1. Linear recursion is a recursion whose order of growth is linear (not branched). Example of Linear recursion is Factorial, defined by fac (n)=n * fac (n-1).

2. Tree recursion will branch to more than one node each step, growing up very quickly. It provides us a flexible ways to solve some logically difficult problem. It can be used to perform a Complete Search (To make the computer to do the trial and error). This recursion type has a quadratic or cubic or more order of growth and therefore, it is not suitable for solving "big" problems. It's limited to small. Examples: solving Tower of Hanoi, Searching (DFS, BFS), Fibonacci number, etc.

Some compilers can make some type of recursion to be iterative. One example is tail-recursion elimination. Tail-recursive is a type of recursive which the recursive call is the last command in that function/procedure. This

Tips on Recursion

1. Recursion is very similar to mathematical induction.
2. You first see how you can solve the base case, for n=0 or for n=1.
3. Then you assume that you know how to solve the problem of size n-1, and you look for a way of obtaining the solution for the problem size n from the solution of size n-1.

When constructing a recursive solution, keep the following questions in mind:

1. How can you define the problem in term of a smaller problem of the same type?
2. How does each recursive call diminish the size of the problem?

3. What instance of the problem can serve as the base case?
4. As the problem size diminishes, will you reach this base case?

Sample of a very standard recursion, Factorial (in Java):

```
static int factorial(int n) {
   if (n==0)
      return 1;
   else
      return n*factorial(n-1);
}
```

Divide and Conquer

This technique use analogy "smaller is simpler". Divide and conquer algorithms try to make problems simpler by dividing it to sub problems that can be solved easier than the main problem and then later it combine the results to produce a complete solution for the main problem.

Divide and Conquer algorithms: Quick Sort, Merge Sort, Binary Search.

Divide & Conquer has 3 main steps:

1. Divide data into parts
2. Find sub-solutions for each of the parts recursively (usually)
3. Construct the final answer for the original problem from the sub-solutions

```
DC(P) {
  if small(P) then
    return Solve(P);
  else {
    divide P into smaller instances P1,...,Pk, k>1;
    Apply DC to each of these subproblems;
    return combine(DC(P1),...,DC(Pk));
  }
}
```

Binary Search, is not actually follow the pattern of the full version of Divide & Conquer, since it never combine the sub problems solution anyway. However, lets use this example to illustrate the capability of this technique.

Binary Search will help you find an item in an array. The naive version is to scan through the array one by one (sequential search). Stopping when we found the item (success) or when we reach the end of array (failure). This is the simplest and the most inefficient way

to find an element in an array. However, you will usually end up using this method because not every array is ordered, you need sorting algorithm to sort them first.

Binary search algorithm is using Divide and Conquer approach to reduce search space and stop when it found the item or when search space is empty.

Pre-requisite to use binary search is the array must be an ordered array, the most commonly used example of ordered array for binary search is searching an item in a dictionary.

Efficiency of binary search is O(log n). This algorithm was named "binary" because it's behavior is to divide search space into 2 (binary) smaller parts).

Optimizing your source code

Your code must be fast. If it cannot be "fast", then it must at least "fast enough". If time limit is 1 minute and your currently program run in 1 minute 10 seconds, you may want to tweak here and there rather than overhaul the whole code again.

Generating vs Filtering

Programs that generate lots of possible answers and then choose the ones that are correct (imagine an 8-queen solver) are filters. Those that hone in exactly on the correct answer without any false starts are generators. Generally, filters are easier (faster) to code and run slower. Do the math to see if a filter is good enough or if you need to try and create a generator.

Pre-Computation / Pre-Calculation

Sometimes it is helpful to generate tables or other data structures that enable the fastest possible lookup of a result. This is called Pre-Computation (in which one trades space for time). One might either compile Pre-Computed data into a program, calculate it when the program starts, or just remember results as you compute them. A program that must translate letters from upper to lower case when they are in upper case can do a very fast table lookup that requires no conditionals, for example. Contest problems often use prime numbers - many times it is practical to generate a long list of primes for use elsewhere in a program.

Decomposition

While there are fewer than 20 basic algorithms used in contest problems, the challenge of combination problems that require a combination of two algorithms for solution is

daunting. Try to separate the cues from different parts of the problem so that you can combine one algorithm with a loop or with another algorithm to solve different parts of the problem independently. Note that sometimes you can use the same algorithm twice on different (independent!) parts of your data to significantly improve your running time.

Symmetries

Many problems have symmetries (e.g., distance between a pair of points is often the same either way you traverse the points). Symmetries can be 2-way, 4-way, 8-way, and more. Try to exploit symmetries to reduce execution time.

For instance, with 4-way symmetry, you solve only one fourth of the problem and then write down the four solutions that share symmetry with the single answer (look out for self-symmetric solutions which should only be output once or twice, of course).

Solving forward vs backward

Surprisingly, many contest problems work far better when solved backwards than when using a frontal attack. Be on the lookout for processing data in reverse order or building an attack that looks at the data in some order or fashion other than the obvious.

CHAPTER 7 MATHEMATICS

Base Number Conversion

Decimal is our most familiar base number, whereas computers are very familiar with binary numbers (octal and hexa too). The ability to convert between bases is important. Refer to mathematics book for this. [2]

TEST YOUR BASE CONVERSION KNOWLEDGE

Solve UVa problems related with base conversion:

343 - What Base Is This?
353 - The Bases Are Loaded
389 - Basically Speaking

Big Mod

Modulo (remainder) is important arithmetic operation and almost every programming language provide this basic operation. However, basic modulo operation is not sufficient if we are interested in finding a modulo of a very big integer, which will be difficult and has tendency for overflow. Fortunately, we have a good formula here:

```
(A*B*C) mod N == ((A mod N) * (B mod N) * (C mod N)) mod N.
```

To convince you that this works, see the following example:

```
(7*11*23) mod 5 is 1; let's see the calculations step by step:

((7 mod 5) * (11 mod 5) * (23 mod 5)) Mod 5 =
((2 * 1 * 3) Mod 5) =
(6 Mod 5) = 1
```

This formula is used in several algorithm such as in Fermat Little Test for primality testing:

R = B^P mod M (**B^P** is a **very very big** number). By using the formula, we can get the correct answer without being afraid of overflow.

This is how to calculate **R** quickly (using Divide & Conquer approach, see exponentiation below for more details):

```
long bigmod(long b,long p,long m) {
  if (p == 0)
    return 1;
  else if (p%2 == 0)
    return square(bigmod(b,p/2,m)) % m; // square(x) = x * x
  else
    return ((b % m) * bigmod(b,p-1,m)) % m;
}
```

TEST YOUR BIG MOD KNOWLEDGE

Solve UVa problems related with Big Mod:

374 - Big Mod
10229 - Modular Fibonacci - plus Fibonacci

Big Integer

Long time ago, 16-bit integer is sufficient: $[-2^{15}$ to $2^{15-1}] \sim$ [-32,768 to 32,767].

When 32-bit machines are getting popular, 32-bit integers become necessary. This data type can hold range from $[-2^{31}$ to $2^{31-1}] \sim$ [2,147,483,648 to 2,147,483,647]. Any integer with 9 or less digits can be safely computed using this data type. If you somehow must use the 10th digit, be careful of overflow.

But man is very greedy, 64-bit integers are created, referred as programmers as long long data type or Int64 data type. It has an awesome range up that can covers 18 digits integers fully, plus the 19th digit partially $[-2^{63}-1$ to $2^{63-1}] \sim$ [9,223,372,036,854,775,808 to 9,223,372,036,854,775,807]. Practically this data type is safe to compute most of standard arithmetic problems, except some problems.

Note: for those who don't know, in C, long long n is read using: scanf("%lld",&n); and unsigned long long n is read using: scanf("%llu",&n); **For 64 bit data:** typedef unsigned long long int64; int64 test_data=64000000000LL

Now, RSA cryptography need 256 bits of number or more.This is necessary, human always want more security right? However, some problem setters in programming contests also follow this trend. Simple problem such as finding n'th factorial will become very very hard once the values exceed 64-bit integer data type. Yes, long double can store very high numbers, but it actually only stores first few digits and an exponent, long double is not precise.

Implement your own arbitrary precision arithmetic algorithm Standard pen and pencil algorithm taught in elementary school will be your tool to implement basic arithmetic operations: addition, subtraction, multiplication and division. More sophisticated

algorithm are available to enhance the speed of multiplication and division. Things are getting very complicated now.

TEST YOUR BIG INTEGER KNOWLEDGE

Solve UVa problems related with Big Integer:

485 - Pascal Triangle of Death
495 - Fibonacci Freeze - plus Fibonacci
10007 - Count the Trees - plus Catalan formula
10183 - How Many Fibs - plus Fibonacci
10219 - Find the Ways ! - plus Catalan formula
10220 - I Love Big Numbers ! - plus Catalan formula
10303 - How Many Trees? - plus Catalan formula
10334 - Ray Through Glasses - plus Fibonacci
10519 - !!Really Strange!!
10579 - Fibonacci Numbers - plus Fibonacci

Carmichael Number

Carmichael number is a number which is not prime but has >= 3 prime factors You can compute them using prime number generator and prime factoring algorithm.

The first 15 Carmichael numbers are:

```
561,1105,1729,2465,2821,6601,8911,10585,15841,29341,41041,46657,52633,6274
5,63973
```

TEST YOUR CARMICHAEL NUMBER KNOWLEDGE

Solve UVa problems related with Carmichael Number:

10006 - Carmichael Number

Catalan Formula

The number of distinct binary trees can be generalized as Catalan formula, denoted as C(n).

```
C(n) = 2n C n / (n+1)
```

If you are asked values of several C(n), it may be a good choice to compute the values bottom up.

Since C(n+1) can be expressed in C(n), as follows:

```
Catalan(n) =

      2n!
---------------
n! * n! * (n+1)

Catalan(n+1) =

           2*(n+1)
-------------------------- =
(n+1)! * (n+1)! * ((n+1)+1)

     (2n+2) * (2n+1) * 2n!
------------------------------ =
(n+1) * n! * (n+1) * n! * (n+2)

     (2n+2) * (2n+1) * 2n!
---------------------------------- =
(n+1) * (n+2) * n! * n! * (n+1)

(2n+2) * (2n+1)
--------------- * Catalan(n)
(n+1) * (n+2)
```

TEST YOUR CATALAN FORMULA KNOWLEDGE

Solve UVa problems related with Catalan Formula:

10007 - Count the Trees - * n! -> number of trees + its permutations
10303 - How Many Trees?

Counting Combinations - C(N,K)

C(N,K) means how many ways that N things can be taken K at a time. This can be a great challenge when N and/or K become very large.

Combination of (N,K) is defined as:

```
     N!
----------
(N-K)!*K!
```

For your information, the exact value of 100! is:

```
93,326,215,443,944,152,681,699,238,856,266,700,490,715,968,264,381,621,46
8,592,963,895,217,599,993,229,915,608,941,463,976,156,518,286,253,697,920
,827,223,758,251,185,210,916,864,000,000,000,000,000,000,000,000
```

So, how to compute the values of C(N,K) when N and/or K is big but the result is guaranteed to fit in 32-bit integer?

Divide by GCD before multiply (sample code in C)

```
long gcd(long a,long b) {
  if (a%b==0) return b; else return gcd(b,a%b);
}
void Divbygcd(long& a,long& b) {
  long g=gcd(a,b);
  a/=g;
  b/=g;
}
long C(int n,int k){
  long numerator=1,denominator=1,toMul,toDiv,i;
  if (k>n/2) k=n-k; /* use smaller k */
  for (i=k;i;i--) {
    toMul=n-k+i;
    toDiv=i;
    Divbygcd(toMul,toDiv);       /* always divide before multiply */
   Divbygcd(numerator,toDiv);

Divbygcd(toMul,denominator);
    numerator*=toMul;
    denominator*=toDiv;
  }
  return numerator/denominator;
}
```

TEST YOUR C(N,K) KNOWLEDGE

Solve UVa problems related with Combinations:

369 - Combinations
530 - Binomial Showdown

Divisors and Divisibility

If d is a divisor of n, then so is n/d, but d & n/d cannot both be greater than sqrt(n).

```
2 is a divisor of 6, so 6/2 = 3 is also a divisor of 6
```

Let N = 25, Therefore no divisor will be greater than sqrt (25) = 5. (Divisors of 25 is 1 & 5 only)

If you keep that rule in your mind, you can design an algorithm to find a divisor better, that's it, no divisor of a number can be greater than the square root of that particular number.

```
If a number N == a^i * b^j * ... * c^k then N has (i+1)*(j+1)*...*(k+1)
divisors.
```

TEST YOUR DIVISIBILITY KNOWLEDGE

Solve UVa problems related with Divisibility:

294 - Divisors

Exponentiation

(Assume pow() function in <math.h> doesn't exist...). Sometimes we want to do exponentiation or take a number to the power n. There are many ways to do that. The standard method is standard multiplication.

Standard multiplication

```
long exponent(long base,long power) {
  long i,result = 1;
  for (i=0; i<power; i++) result *= base;
  return result;}
```

This is 'slow', especially when power is big - O(n) time. It's better to use divide & conquer.

Divide & Conquer exponentiation

```
long square(long n) { return n*n; }
long fastexp(long base,long power) {
  if (power == 0)
    return 1;
  else if (power%2 == 0)
    return square(fastexp(base,power/2));
  else
    return base * (fastexp(base,power-1));
}
```

Using built in formula, a^n = exp(log(a)*n) or pow(a,n)

```
#include <stdio.h>
#include <math.h>

void main() {
  printf("%lf\n",exp(log(8.0)*1/3.0));
  printf("%lf\n",pow(8.0,1/3.0));
}
```

TEST YOUR EXPONENTIATION KNOWLEDGE

Solve UVa problems related with Exponentiation:

113 - Power of Cryptography

Factorial

Factorial is naturally defined as Fac(n) = n * Fac(n-1).

```
Example:

Fac(3) = 3 * Fac(2)
Fac(3) = 3 * 2 * Fac(1)
Fac(3) = 3 * 2 * 1 * Fac(0)
Fac(3) = 3 * 2 * 1 * 1
Fac(3) = 6
```

Iterative version of factorial

```
long FacIter(int n) {
  int i,result = 1;
  for (i=0; i<n; i++) result *= i;
  return result;
}
```

This is the best algorithm to compute factorial. O(n).

Fibonacci

Fibonacci numbers was invented by Leonardo of Pisa (Fibonacci). He defined fibonacci numbers as a growing population of immortal rabbits. Series of Fibonacci numbers are: 1,1,2,3,5,8,13,21,...

Recurrence relation for Fib(n):

```
Fib(0) = 0
Fib(1) = 1
Fib(n) = Fib(n-1) + Fib(n-2)
```

Iterative version of Fibonacci (using Dynamic Programming)

```
int fib(int n) {
  int a=1,b=1,i,c;
  for (i=3; i<=n; i++) {
    c = a+b;
    a = b;
    b = c;
  }
  return a;
}
```

This algorithm runs in linear time O(n).

Quick method to quickly compute Fibonacci, using Matrix property.

```
Divide_Conquer_Fib(n) {
    i = h = 1;
    j = k = 0;
    while (n > 0) {
      if (n%2 == 1) { // if n is odd
        t = j*h;
        j = i*h + j*k + t;
       i = i*k + t;
       }
      t = h*h;
      h = 2*k*h + t;
      k = k*k + t;
      n = (int) n/2;
    }  return j;}
```

This runs in O(log n) time.

TEST YOUR FIBONACCI KNOWLEDGE

Solve UVa problems related with Fibonacci:
495 - Fibonacci Freeze
10183 - How Many Fibs
10229 - Modular Fibonacci
10334 - Ray Through Glasses
10450 - World Cup Noise
10579 - Fibonacci Numbers

Greatest Common Divisor (GCD)

As it name suggest, Greatest Common Divisor (GCD) algorithm finds the greatest common divisor between two number a and b. GCD is a very useful technique, for example to reduce rational numbers into its smallest version (3/6 = 1/2).The best GCD algorithm so far is Euclid's Algorithm. Euclid found this interesting mathematical equality. GCD(a,b) = GCD (b,(a mod b)). Here is the fastest implementation of GCD algorithm

```
int GCD(int a,int b) {
  while (b > 0) {
    a = a % b;
    a ^= b;    b ^= a;    a ^= b;  }  return a;
}
```

Lowest Common Multiple (LCM)

Lowest Common Multiple and Greatest Common Divisor (GCD) are two important number properties, and they are interrelated. Even though GCD is more often used than LCM, it is useful to learn LCM too.

```
LCM (m,n) = (m * n) / GCD (m,n)

LCM (3,2) = (3 * 2) / GCD (3,2)
LCM (3,2) = 6 / 1
LCM (3,2) = 6
```

Application of LCM -> to find a synchronization time between two traffic light, if traffic light A display green color every 3 minutes and traffic light B display green color every 2 minutes, then every 6 minutes, both traffic light will display green color at the same time.

Mathematical Expressions

There are three types of mathematical/algebraic expressions. They are Infix, Prefix, & Postfix expressions.

Infix expression grammar:
<infix> = <identifier> | <infix><operator><infix>
<operator> = + | - | * | / <identifier> = a | b | .... | z

Infix expression example: (1 + 2) => 3, the operator is in the middle of two operands. Infix expression is the normal way human compute numbers.

Prefix expression grammar:
<prefix> = <identifier> | <operator><prefix><prefix>
<operator> = + | - | * | / <identifier> = a | b | .... | z

Prefix expression example: (+ 1 2) => (1 + 2) = 3, the operator is the first item.
One programming language that use this expression is Scheme language.
The benefit of prefix expression is it allows you write: (1 + 2 + 3 + 4)
like this (+ 1 2 3 4), this is simpler.

Postfix expression grammar:
<postfix> = <identifier> | <postfix><postfix><operator>
<operator> = + | - | * | / <identifier> = a | b | .... | z

Postfix expression example: (1 2 +) => (1 + 2) = 3, the operator is the last item.

Postfix Calculator

Computer do postfix calculation better than infix calculation. Therefore when you compile your program, the compiler will convert your infix expressions (most programming language use infix) into postfix, the computer-friendly version.

Why do computer like Postfix better than Infix?

It's because computer use stack data structure, and postfix calculation can be done easily using stack.

Infix to Postfix conversion

To use this very efficient Postfix calculation, we need to convert our Infix expression into Postfix. This is how we do it:

Example:

Infix statement: ((4 - (1 + 2 * (6 / 3) - 5))). This is what the algorithm will do, look carefully at the steps.

Expression	Stack (bottom to top)	Postfix expression
((4-(1+2*(6/3)-5)))	(	
((4-(1+2*(6/3)-5)))	((	
((4-(1+2*(6/3)-5)))	((	4
((4-(1+2*(6/3)-5)))	((-	4

((4-(1+2*(6/3)-5)))	((-(	4
((4-(1+2*(6/3)-5)))	((-(	41
((4-(1+2*(6/3)-5)))	((-(+	41
((4-(1+2*(6/3)-5)))	((-(+	412
((4-(1+2*(6/3)-5)))	((-(+*	412
((4-(1+2*(6/3)-5)))	((-(+*(	412
((4-(1+2*(6/3)-5)))	((-(+*(	4126
((4-(1+2*(6/3)-5)))	((-(+*(/	4126
((4-(1+2*(6/3)-5)))	((-(+*(/	41263
((4-(1+2*(6/3)-5)))	((-(+*	41263/
((4-(1+2*(6/3)-5)))	((-(-	41263/*+
((4-(1+2*(6/3)-5)))	((-(-	41263/*+5
((4-(1+2*(6/3)-5)))	((-	41263/*+5-
((4-(1+2*(6/3)-5)))	(	41263/*+5--
((4-(1+2*(6/3)-5)))		41263/*+5--

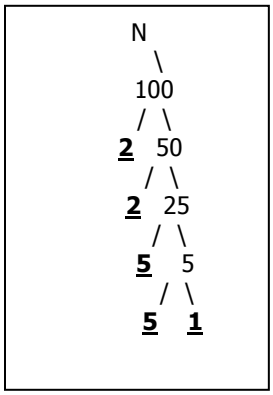
TEST YOUR INFIX->POSTFIX KNOWLEDGE

Solve UVa problems related with postfix conversion:

727 - Equation

Prime Factors

All integers can be expressed as a product of primes and these primes are called **prime factors** of that number.



Exceptions:

For negative integers, multiply by -1 to make it positive again. For -1,0, and 1, no prime factor. (by definition...)

Standard way

Generate a prime list again, and then check how many of those primes can divide n.This is very slow. Maybe you can write a very efficient code using this algorithm, but there is another algorithm that is much more effective than this.

Creative way, using Number Theory

1. Don't generate prime, or even checking for primality.
2. Always use **stop-at-sqrt** technique
3. Remember to check repetitive prime factors, example=20->2*2*5, don't count 2 twice. We want distinct primes (however, several problems actually require you to count these multiples)
4. Make your program use constant memory space, no array needed.
5. Use the definition of prime factors wisely, this is the key idea. A number can always be divided into a prime factor and another prime factor or another number. This is called the factorization tree.

From this factorization tree, we can determine these following properties:

1. If we take out a prime factor (F) from any number N, then the N will become smaller and smaller until it become 1, we stop here.
2. This smaller N can be a prime factor or it can be another smaller number, we don't care, all we have to do is to repeat this process until N=1.

TEST YOUR PRIME FACTORS KNOWLEDGE
Solve UVa problems related with prime factors: 583 - Prime Factors

Prime Numbers

Prime numbers are important in Computer Science (For example: Cryptography) and finding prime numbers in a given interval is a "tedious" task. Usually, we are looking for big (very big) prime numbers therefore searching for a better prime numbers algorithms will never stop.

1. Standard prime testing

```
int is_prime(int n) {
  for (int i=2; i<=(int) sqrt(n); i++) if (n%i == 0) return 0;
  return 1;
}void main() {
  int i,count=0;
  for (i=1; i<10000; i++) count += is_prime(i);
  printf("Total of %d primes\n",count);
}
```